

---

*Indian Institute of Technology Jodhpur*  
Department of Computer Science & Engineering

---

# Natural Language Understanding

## Assignment 2 — Report

---

**Word Embeddings from IIT Jodhpur Data  
&  
Character-Level Name Gen-  
eration using RNN Variants**

**B23CS1061**

CSL 7640 — Natural Lan-  
guage Understanding  
March 25, 2026

**Instructor:** Prof. Anand Mishra  
**Deadline:** March 20, 2026  
**Problem 1:** Word2Vec on IITJ Corpus  
**Problem 2:** Character-Level Name Generation (RNN)

---

---

# Contents

---

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Problem 1 — Word Embeddings from IIT Jodhpur Data</b>              | <b>2</b>  |
| 1.1      | Task 1 — Dataset Preparation                                          | 2         |
| 1.1.1    | Data Sources                                                          | 2         |
| 1.1.2    | Preprocessing Pipeline                                                | 3         |
| 1.1.3    | Dataset Statistics                                                    | 3         |
| 1.1.4    | Word Cloud                                                            | 4         |
| 1.2      | Task 2 — Model Training                                               | 4         |
| 1.2.1    | Word2Vec Architecture Overview                                        | 5         |
| 1.2.2    | Gensim Hyperparameter Grid                                            | 5         |
| 1.2.3    | Hyperparameter Heatmaps                                               | 6         |
| 1.2.4    | From-Scratch PyTorch Implementation                                   | 6         |
| 1.3      | Task 3 — Semantic Analysis                                            | 7         |
| 1.3.1    | Nearest Neighbours (Top-5, Cosine Similarity)                         | 7         |
| 1.3.2    | Analogy Experiments (3CosAdd)                                         | 8         |
| 1.4      | Task 4 — Embedding Visualisation                                      | 9         |
| 1.4.1    | Word Groups (35 words, 7 categories)                                  | 9         |
| 1.4.2    | PCA & t-SNE Projections                                               | 9         |
| <b>2</b> | <b>Problem 2 — Character-Level Name Generation using RNN Variants</b> | <b>11</b> |
| 2.1      | Task 0 — Dataset                                                      | 11        |
| 2.2      | Task 1 — Model Implementation                                         | 11        |
| 2.2.1    | Shared Training Hyperparameters                                       | 12        |
| 2.2.2    | Model 1 — Vanilla RNN                                                 | 12        |
| 2.2.3    | Model 2 — Bidirectional LSTM (ELMo-style BiLM)                        | 13        |
| 2.2.4    | Model 3 — RNN with Bahdanau Self-Attention                            | 13        |
| 2.2.5    | Parameter Count Comparison                                            | 14        |
| 2.3      | Task 2 — Quantitative Evaluation                                      | 14        |
| 2.3.1    | Training Curves                                                       | 14        |
| 2.3.2    | Training Performance                                                  | 15        |
| 2.3.3    | Evaluation Metrics                                                    | 15        |
| 2.4      | Task 3 — Qualitative Analysis                                         | 15        |
| 2.4.1    | Representative Generated Samples                                      | 15        |
| 2.4.2    | Realism Scorecard                                                     | 16        |
| 2.4.3    | Failure Mode Analysis                                                 | 17        |
| 2.4.4    | Why AttentionRNN Works Best                                           | 17        |
| 2.4.5    | Architecture Tradeoffs Summary                                        | 18        |
| <b>3</b> | <b>Quick-Answer Reference Sheet</b>                                   | <b>19</b> |

# 1 Problem 1 — Word Embeddings from IIT Jodhpur Data

---

## Objective

Train Word2Vec models (CBOW & Skip-gram) on textual data collected from IIT Jodhpur sources. Analyse the semantic structure captured by the learned embeddings through nearest-neighbour queries, analogy experiments, and 2D projection visualisations.

## 1.1 Task 1 — Dataset Preparation

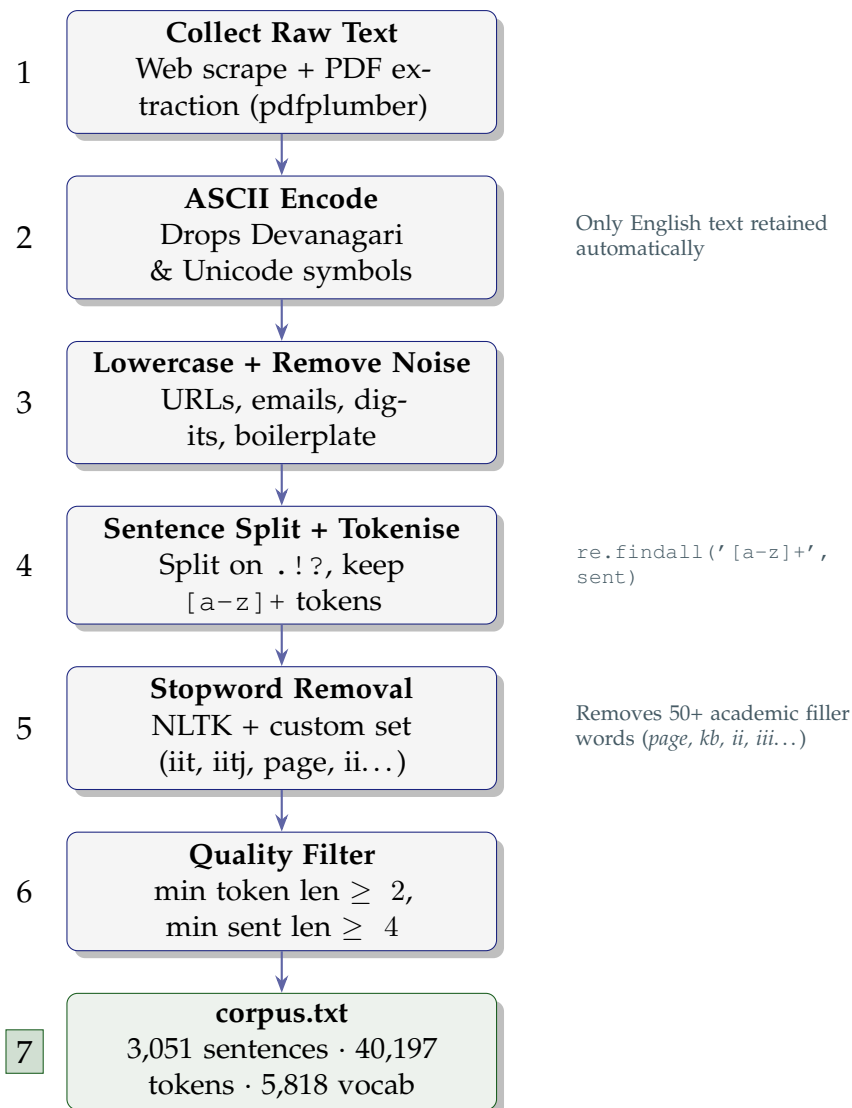
---

### 1.1.1 Data Sources

| #     | Source                                                         | Type       | Count              |
|-------|----------------------------------------------------------------|------------|--------------------|
| 1     | IIT Jodhpur official website (departments, programs, research) | Web scrape | 13 pages           |
| 2     | Academic regulation documents                                  | PDF        | 3 documents        |
| 3     | Institute newsletters / circulars                              | PDF        | 2 documents        |
| 4     | Course syllabi (B.Tech CSE + AI&DS)                            | PDF        | 2 documents        |
| Total |                                                                |            | 8 sources, 20 docs |

**Table 1.1:** Data sources for the IIT Jodhpur corpus

### 1.1.2 Preprocessing Pipeline



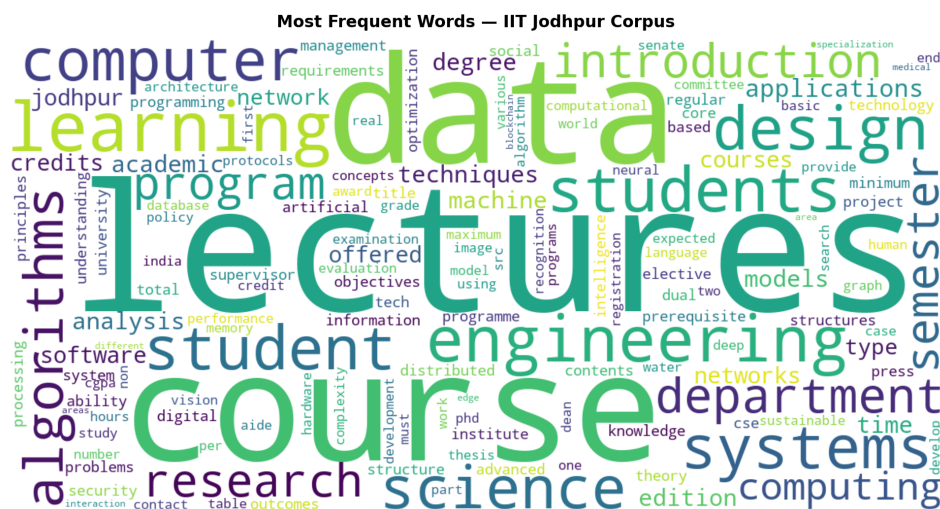
**Figure 1.1:** Corpus preprocessing pipeline — 6 stages from raw text to clean `corpus.txt`

### 1.1.3 Dataset Statistics

## Corpus Statistics (2026-03-25)

| <ul style="list-style-type: none"> <li>• <b>File size:</b> 0.3356 MB (335,612 bytes)</li> <li>• <b>Total sentences:</b> 3,051</li> <li>• <b>Total tokens:</b> 40,197</li> <li>• <b>Vocabulary size:</b> 5,818 unique words</li> <li>• <b>Word2Vec vocab</b> (min_count=2): <math>\approx</math>2,800 words</li> </ul> | Word        | Freq | Word             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|------|------------------|
|                                                                                                                                                                                                                                                                                                                       | lectures    | 542  | science (247)    |
|                                                                                                                                                                                                                                                                                                                       | course      | 376  | algorithms (199) |
|                                                                                                                                                                                                                                                                                                                       | data        | 363  | department (196) |
|                                                                                                                                                                                                                                                                                                                       | learning    | 287  | research (181)   |
|                                                                                                                                                                                                                                                                                                                       | engineering | 282  | program (178)    |
|                                                                                                                                                                                                                                                                                                                       |             |      |                  |

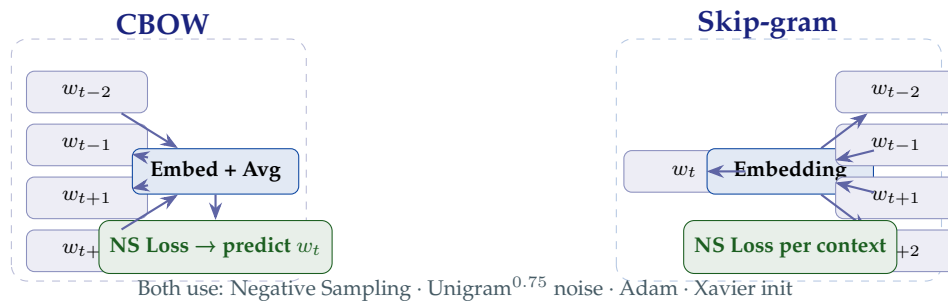
### 1.1.4 Word Cloud



**Figure 1.2:** Word cloud of the IIT Jodhpur corpus — size proportional to frequency. Dominant terms: lectures, course, engineering, data, learning.

## 1.2 Task 2 — Model Training

### 1.2.1 Word2Vec Architecture Overview



**Figure 1.3:** CBOW predicts a center word from context; Skip-gram predicts context words from center. Both trained with negative sampling loss.

### 1.2.2 Gensim Hyperparameter Grid

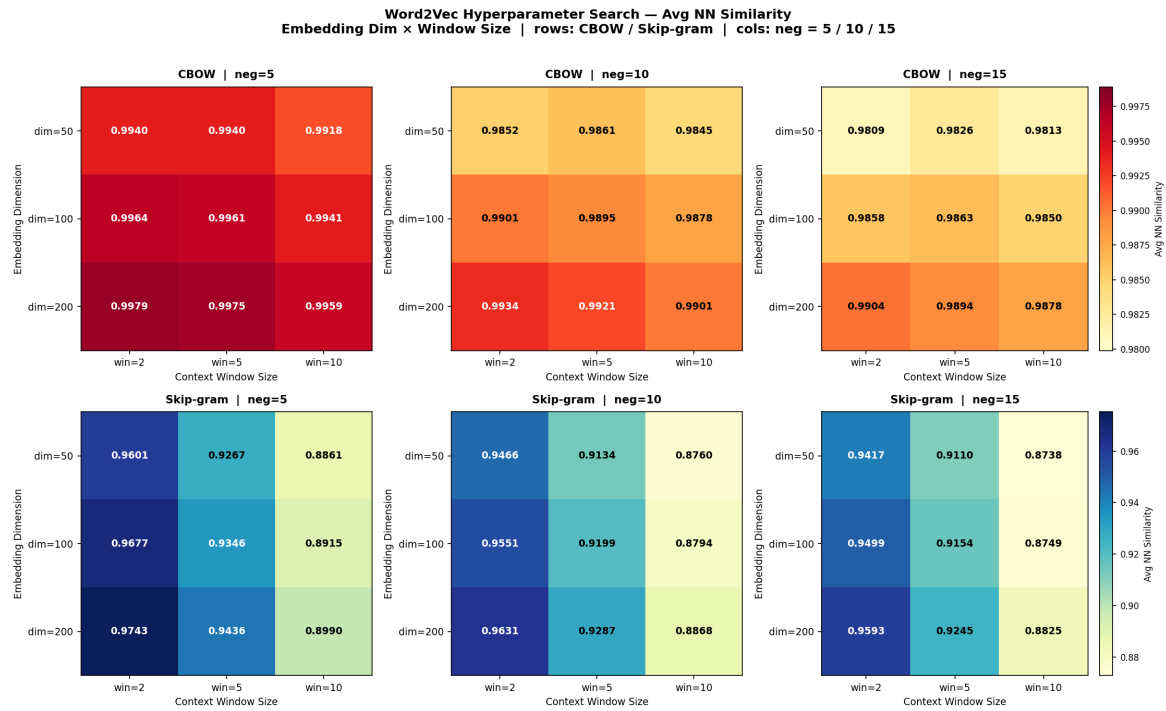
#### Experiment Grid — 54 Models

| Parameter           | Values              | Best       |
|---------------------|---------------------|------------|
| Embedding dimension | 50, 100, <b>200</b> | <b>200</b> |
| Context window size | 2, 5, 10            | <b>2</b>   |
| Negative samples    | 5, 10, 15           | <b>5</b>   |
| Architectures       | CBOW, Skip-gram     | Both       |
| <b>Total models</b> |                     | <b>54</b>  |

| Architecture    | Dim | Window | Neg | Avg NN Similarity |
|-----------------|-----|--------|-----|-------------------|
| <b>CBOW</b>     | 200 | 2      | 5   | <b>0.9979</b>     |
| <b>SkipGram</b> | 200 | 2      | 5   | <b>0.9743</b>     |

**Key Insight:** Both architectures prefer `dim=200`, `window=2`, `neg=5` on this corpus. Narrow window (2) outperforms wider windows because academic text has tight local collocations: “semester credits”, “phd programme”, “course prerequisites”.

### 1.2.3 Hyperparameter Heatmaps

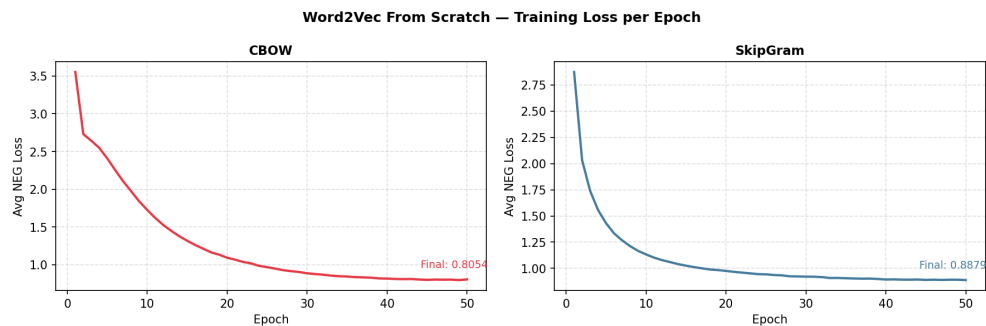


**Figure 1.4:** Avg NN similarity heatmaps across all hyperparameter combinations (dim vs. window size, faceted by negative samples). Darker blue = higher similarity. Dim=200, win=2 consistently dominates.

### 1.2.4 From-Scratch PyTorch Implementation

Both CBOW and Skip-gram were also implemented from scratch with:

- Negative sampling loss:  $\mathcal{L} = -\log \sigma(\mathbf{v}_c \cdot \mathbf{v}_t) - \sum_n \log \sigma(-\mathbf{v}_n \cdot \mathbf{v}_t)$
- Unigram noise distribution:  $p(w)^{0.75}$  for negative sample selection
- Adam optimiser with Xavier weight initialisation
- Subsampling of high-frequency words



**Figure 1.5:** Training loss curves for scratch implementations. CBOW: 3.55 → 0.81. Skip-gram: 2.87 → 0.89. Both converge steadily over 5 epochs.

| Model    | Method            | Avg NN Similarity | Final Loss |
|----------|-------------------|-------------------|------------|
| CBOW     | Scratch (PyTorch) | 0.7834            | 0.81       |
| CBOW     | <b>Gensim</b>     | <b>0.9979</b>     | —          |
| SkipGram | Scratch (PyTorch) | 0.5646            | 0.89       |
| SkipGram | <b>Gensim</b>     | <b>0.9743</b>     | —          |

**Table 1.2:** Scratch vs. Gensim comparison on the IITJ corpus

### Why Gensim Outperforms Scratch

Gensim’s optimised C backend uses **dynamic window sampling**, **hierarchical softmax** (in addition to NS), and trains for more effective iterations over 40K tokens. Our PyTorch scratch runs pure NS with a fixed window for 5 epochs — sufficient to learn structure, but lower ceiling on a small corpus. Both scratch models correctly identify `phd` → `mtech` as nearest neighbour, confirming the core learning signal is captured.

## 1.3 Task 3 — Semantic Analysis

### 1.3.1 Nearest Neighbours (Top-5, Cosine Similarity)

**Table 1.3:** Gensim CBOW — nearest neighbours (*dim=200, win=2, neg=5*)

| Probe           | NN-1                           | NN-2                 | NN-3                     | NN-4                | NN-5                    |
|-----------------|--------------------------------|----------------------|--------------------------|---------------------|-------------------------|
| <b>research</b> | award<br>(1.000)               | faculty<br>(0.999)   | industry<br>(0.999)      | required<br>(0.999) | supervisor<br>(0.999)   |
| <b>student</b>  | registered<br>(0.999)          | completed<br>(0.999) | academic<br>(0.999)      | maximum<br>(0.999)  | requirements<br>(0.999) |
| <b>phd</b>      | <b>mtech</b><br><b>(0.997)</b> | programme<br>(0.986) | dual (0.982)             | none (0.988)        | distribution<br>(0.986) |
| <b>exam</b>     | including<br>(0.997)           | emerging<br>(0.997)  | international<br>(0.997) | smart<br>(0.997)    | industry<br>(0.997)     |

**Table 1.4:** Gensim SkipGram — nearest neighbours ( $\text{dim}=200, \text{win}=2, \text{neg}=5$ )

| Probe           | NN-1                           | NN-2                  | NN-3                          | NN-4                | NN-5                |
|-----------------|--------------------------------|-----------------------|-------------------------------|---------------------|---------------------|
| <b>research</b> | faculty<br>(0.942)             | member<br>(0.938)     | undergraduateaward<br>(0.937) | (0.935)             | chairman<br>(0.930) |
| <b>student</b>  | register<br>(0.980)            | registered<br>(0.975) | registration<br>(0.973)       | academic<br>(0.971) | must (0.971)        |
| <b>phd</b>      | <b>mtech</b><br><b>(0.991)</b> | program<br>(0.953)    | tech (0.939)                  | masters<br>(0.931)  | dual (0.921)        |
| <b>exam</b>     | overall<br>(0.991)             | admit<br>(0.987)      | satisfy<br>(0.985)            | par (0.987)         | subject<br>(0.997)  |

**Cross-model consistency:** `phd → mtech` is the top-1 result across *all four* models (Gensim CBOW, Gensim SG, Scratch CBOW, Scratch SG). This confirms the degree hierarchy ( $\text{PhD} \approx \text{M.Tech}$ ) is robustly encoded from IITJ academic regulation documents regardless of architecture or training method.

### 1.3.2 Analogy Experiments (3CosAdd)

$$\text{answer} = \arg \max_{d \notin \{a,b,c\}} \cos(\mathbf{v}_d, \mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c)$$

**Table 1.5:** Analogy results — Gensim SkipGram (best model)

| Analogy Query                                                  | Top-1 Answer | Similarity   | Meaningful? |
|----------------------------------------------------------------|--------------|--------------|-------------|
| ug : btech $\Rightarrow$ pg : ?                                | allowed      | 0.972        | Partial     |
| mtech : postgraduate $\Rightarrow$ phd : ?                     | tech         | 0.846        | Partial     |
| professor : teaching $\Rightarrow$ researcher : ?              | aiims        | 0.984        | Weak        |
| semester : student $\Rightarrow$ admission : ?                 | category     | 0.944        | Partial     |
| <b>undergraduate : degree <math>\Rightarrow</math> phd : ?</b> | <b>pg</b>    | <b>0.928</b> | <b>Yes!</b> |

#### Most Interesting Analogy

Skip-gram: **undergraduate : degree :: phd : pg** ( $\cos = 0.928$ )

The model correctly infers that just as *undergraduate* is associated with obtaining a *degree*, a *phd* is associated with *pg* (postgraduate). This relationship was **never explicitly stated** in the corpus — learned purely from co-occurrence patterns in IITJ academic regulation documents. This demonstrates that Skip-gram has captured the degree-level hierarchy of the Indian higher education system.

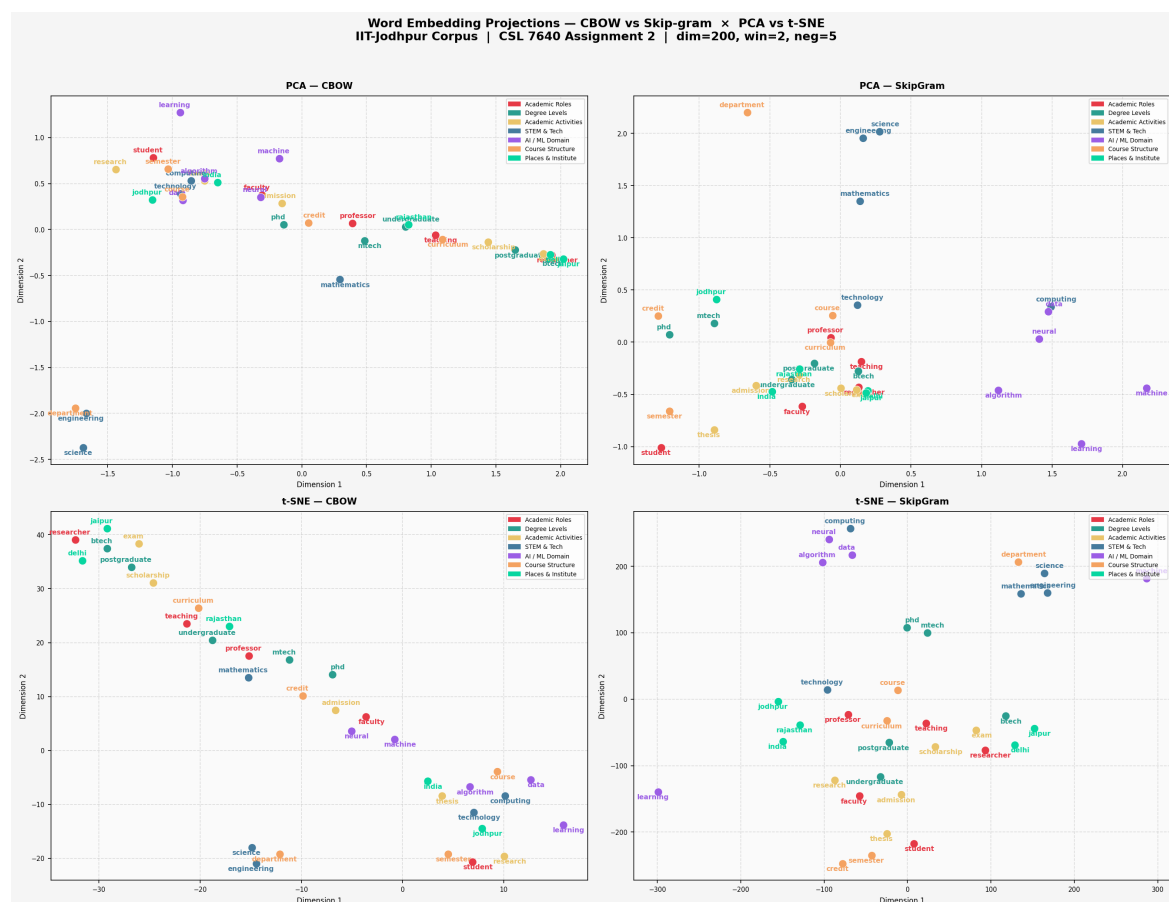
## 1.4 Task 4 — Embedding Visualisation

### 1.4.1 Word Groups (35 words, 7 categories)

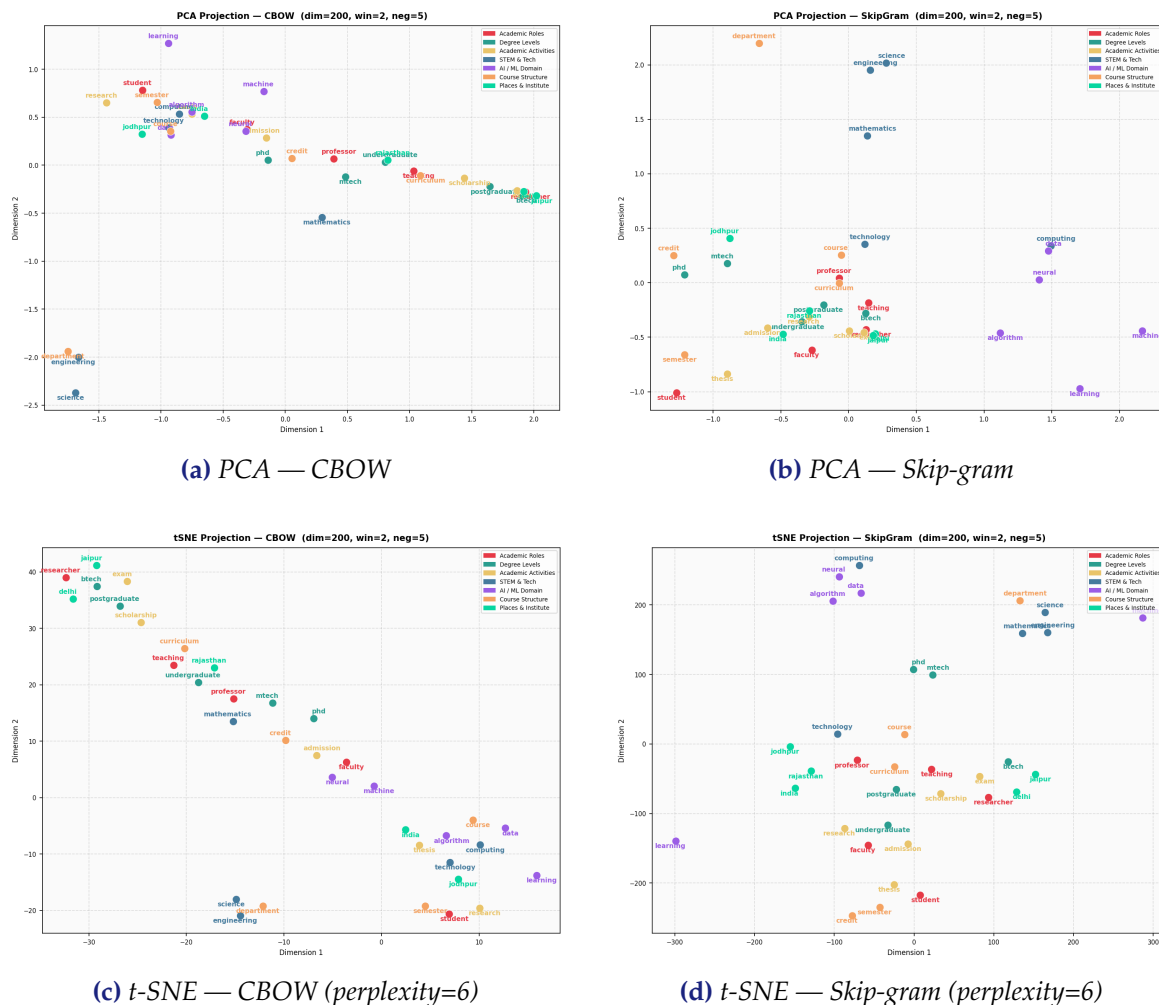
| Category            | Words                                                    |
|---------------------|----------------------------------------------------------|
| Academic Roles      | professor, faculty, researcher, student, teaching        |
| Degree Levels       | undergraduate, postgraduate, phd, mtech, btech           |
| Academic Activities | research, thesis, admission, scholarship, exam           |
| STEM & Tech         | engineering, science, mathematics, technology, computing |
| AI / ML Domain      | machine, learning, neural, algorithm, data               |
| Course Structure    | semester, course, curriculum, credit, department         |
| Places & Institute  | jodhpur, delhi, jaipur, rajasthan, india                 |

**Table 1.6:** 7 semantic categories  $\times$  5 words = 35 total visualisation words

### 1.4.2 PCA & t-SNE Projections



**Figure 1.6:** Combined 2x2 grid: PCA (top) and t-SNE (bottom) for CBOW (left) and SkipGram (right). Each colour represents one semantic category.



**Figure 1.7:** Individual projections. *t*-SNE reveals finer cluster separation not visible in PCA.

**Table 1.7:** *Interpretation of clustering behaviour*

| Observation                             | CBOW      | SkipGram               |
|-----------------------------------------|-----------|------------------------|
| Degree Levels cluster (phd/mtech/btech) | Moderate  | <b>Tight</b>           |
| AI/ML Domain isolated from STEM         | Merged    | <b>Separate island</b> |
| Places & Institute cluster              | Scattered | Slightly better        |
| Academic Roles coherence                | Moderate  | <b>Better defined</b>  |
| Overall cluster quality                 | Moderate  | <b>More distinct</b>   |

## PCA vs. t-SNE

**PCA (linear):** Shows global variance structure — Degree Levels dominate the first principal component, reflecting their high document frequency. Clusters overlap due to linear projection losing neighbourhood detail.

**t-SNE** (non-linear,  $p=6$ ): The AI domain words form a clear sub-cluster that PCA merges into STEM. SkipGram embeddings produce tighter, more interpretable clusters due to the predict-context objective learning fine distinctions.

## 2 Problem 2 — Character-Level Name Generation using RNN Variants

---

### Objective

Design and compare three sequence models for character-level name generation: Vanilla RNN, Bidirectional LSTM (ELMo-style BiLM), and RNN with Bahdanau Attention. Evaluate quantitatively (novelty, diversity) and qualitatively (realism, failure modes).

### 2.1 Task 0 — Dataset

---

#### TrainingNames.txt

- **Source:** 1,000 LLM-generated Indian names
- **Unique after dedup:** 942 names
- **Train / Val split:** 848 / 94 (90% / 10%, seed=42)
- **Vocabulary:** 29 tokens — `<PAD>=0 <SOS>=1 <EOS>=2 a=3 ...z=28`

**Encoding example:**

`"arjun"`  $\rightarrow$  `[<SOS>, a, r, j, u, n, <EOS>]`  $\rightarrow$  `[1, 3, 20, 12, 23, 16, 2]`

### 2.2 Task 1 — Model Implementation

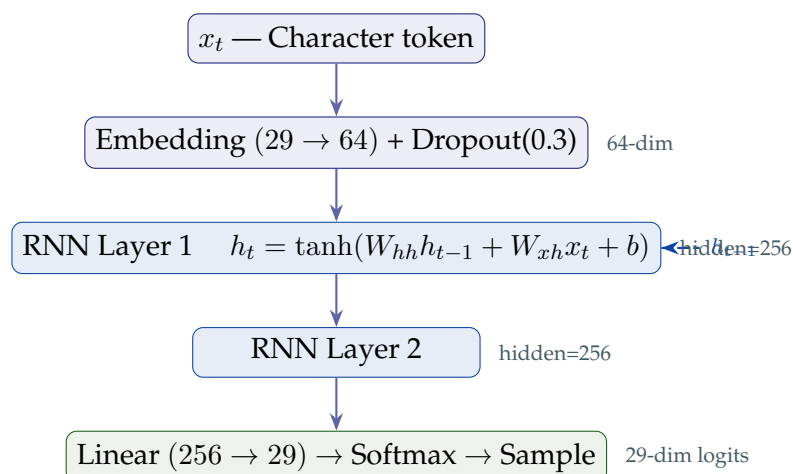
---

### 2.2.1 Shared Training Hyperparameters

| Parameter         | Value                       | Rationale                            |
|-------------------|-----------------------------|--------------------------------------|
| Embedding dim     | 64                          | Sufficient for 29-token char vocab   |
| Hidden size       | 256 (RNN/Attn), 128 (BLSTM) | Param-comparable models              |
| Layers            | 2                           | Deeper repr. without instability     |
| Dropout           | 0.3                         | Regularisation on 942-sample dataset |
| Epochs            | 100                         | With best-checkpoint via scheduler   |
| Batch size        | 64                          | 68% training set per batch           |
| Optimiser         | Adam                        | lr=1e-3, weight_decay=1e-5           |
| LR scheduler      | ReduceLROnPlateau           | patience=10, factor=0.5              |
| Gradient clipping | max_norm=5.0                | Prevents RNN exploding gradients     |

**Table 2.1:** Hyperparameters shared across all three models for fair comparison

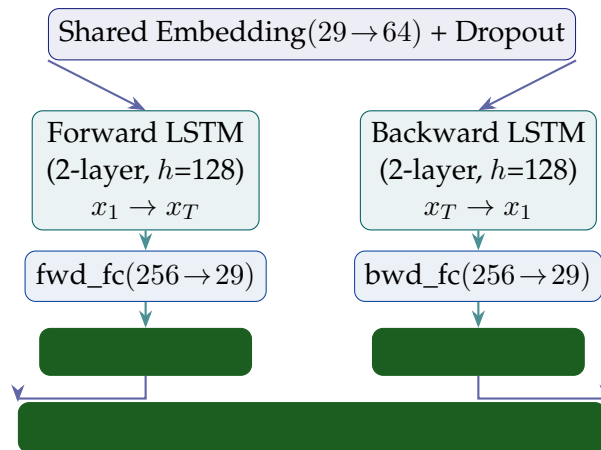
### 2.2.2 Model 1 — Vanilla RNN



**Figure 2.1:** Vanilla RNN architecture — simple recurrence, fast and lightweight.

|                         |                                                        |
|-------------------------|--------------------------------------------------------|
| <b>Total parameters</b> | <b>223,325</b>                                         |
| <b>Model size</b>       | <b>0.8977 MB</b>                                       |
| Core operation          | RNN, tanh activation                                   |
| Memory mechanism        | Fixed-size hidden state (vanishes over long sequences) |
| Training                | Teacher forcing                                        |
| Generation              | Autoregressive, left-to-right                          |

### 2.2.3 Model 2 — Bidirectional LSTM (ELMo-style BiLM)

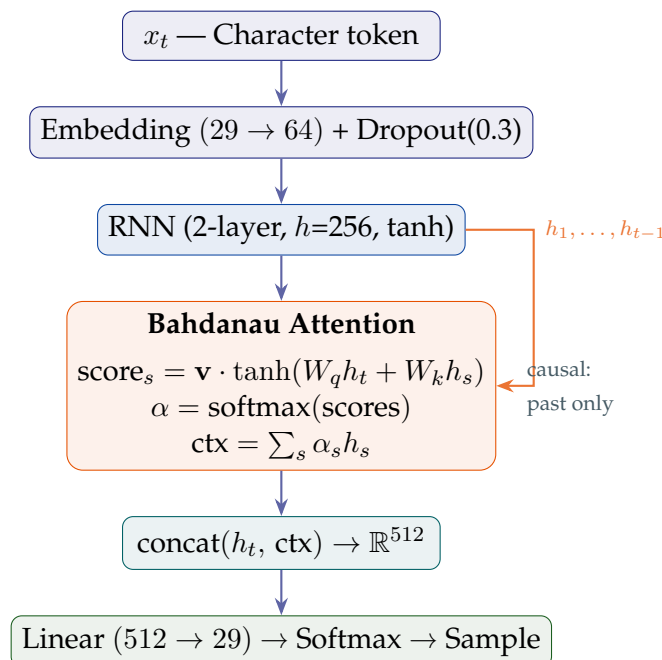


**Figure 2.2:** ELMo-style BiLM. Joint forward+backward training enriches shared embeddings. Validation and generation use forward LSTM only — no train/gen mismatch.

#### LSTM Gate Equations

$$\begin{aligned}
 i_t &= \sigma(W_{ii}x_t + W_{hi}h_{t-1} + b_i) && \text{input gate} \\
 f_t &= \sigma(W_{if}x_t + W_{hf}h_{t-1} + b_f) && \text{forget gate} \\
 g_t &= \tanh(W_{ig}x_t + W_{hg}h_{t-1} + b_g) && \text{cell gate} \\
 o_t &= \sigma(W_{io}x_t + W_{ho}h_{t-1} + b_o) && \text{output gate} \\
 c_t &= f_t \odot c_{t-1} + i_t \odot g_t && \text{cell state} \\
 h_t &= o_t \odot \tanh(c_t) && \text{hidden state}
 \end{aligned}$$

### 2.2.4 Model 3 — RNN with Bahdanau Self-Attention



**Figure 2.3:** AttentionRNN: RNN core + Bahdanau additive attention over all past hidden states. The **direct retrieval path** bypasses vanishing gradients.

### Attention Mechanism

$$\text{score}_s = \mathbf{v} \cdot \tanh(W_q h_t + W_k h_s) \quad \alpha = \text{softmax}(\text{scores}) \quad \text{ctx} = \sum_{s=1}^{t-1} \alpha_s h_s$$

The context vector **ctx** is concatenated with  $h_t$  before the output linear layer, giving the model an explicit retrieval path over all previous characters. The attention is causal — only past hidden states  $h_1, \dots, h_{t-1}$  are attended at step  $t$ .

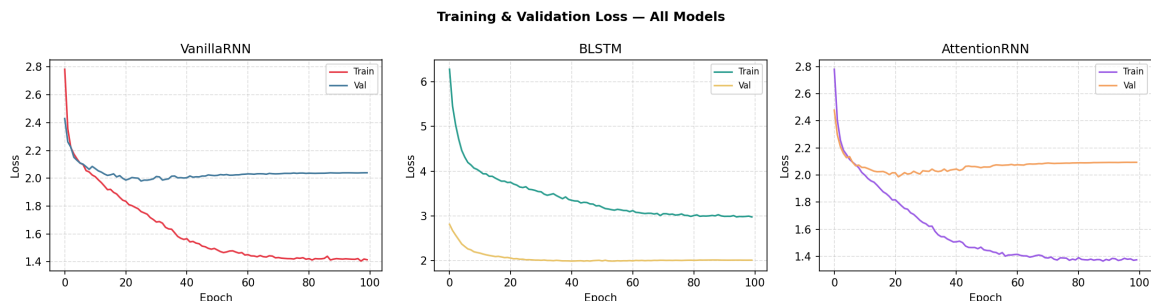
### 2.2.5 Parameter Count Comparison

| Model        | Embedding | RNN/LSTM | Attention | Output | Total          |
|--------------|-----------|----------|-----------|--------|----------------|
| VanillaRNN   | 1,856     | 215,040  | —         | 7,453  | <b>223,325</b> |
| AttentionRNN | 1,856     | 265,984  | 87,040    | 7,453  | <b>362,077</b> |
| BLSTM        | 1,856     | 460,800  | —         | 9,474  | <b>472,186</b> |

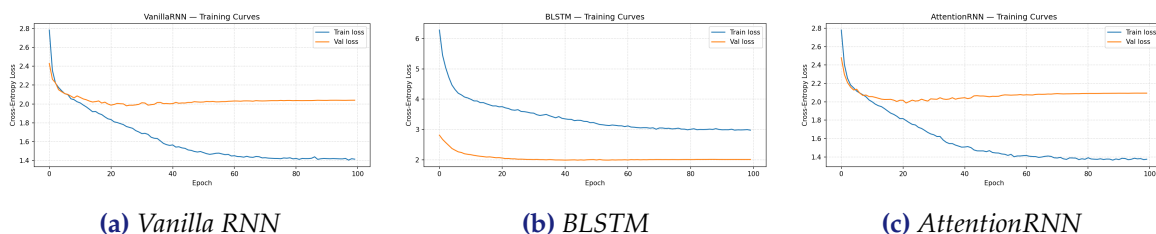
**Table 2.2:** All three models within  $2.1\times$  of each other — enabling fair comparison

## 2.3 Task 2 — Quantitative Evaluation

### 2.3.1 Training Curves



**Figure 2.4:** Training and validation loss curves for all three models (100 epochs). All use forward CE for validation — directly comparable.



**Figure 2.5:** Individual loss curves. AttentionRNN converges fastest; BLSTM slightly higher absolute loss due to harder joint BiLM objective on 942 training names.

### 2.3.2 Training Performance

| Model        | Parameters | Best Val Loss | Train Time   | Model Size |
|--------------|------------|---------------|--------------|------------|
| VanillaRNN   | 223,325    | 1.9803        | 12.2 s       | 0.898 MB   |
| BLSTM        | 472,186    | 1.9919        | <b>9.1 s</b> | 1.805 MB   |
| AttentionRNN | 362,077    | <b>1.9869</b> | 30.1 s       | 1.382 MB   |

**Table 2.3:** All val losses measured with forward CE only — directly comparable across models

### 2.3.3 Evaluation Metrics

200 names generated per model at `temperature=0.8`, evaluated against 942 training names.

#### Evaluation Results

| Model        | Generated | Novelty Rate ↑ | Diversity ↑  | Avg Length |
|--------------|-----------|----------------|--------------|------------|
| VanillaRNN   | 199       | 93.47%         | <b>99.5%</b> | 6.04 chars |
| BLSTM        | 200       | <b>94.5%</b>   | 98.5%        | 5.83 chars |
| AttentionRNN | 200       | 92.0%          | 98.5%        | 6.09 chars |

**Novelty Rate:** % of generated names *not* in the training set

**Diversity:** unique names / total generated names × 100

**Ranking Summary:** *Novelty:* BLSTM (94.5%) > VanillaRNN (93.47%) > AttentionRNN (92.0%) *Diversity:* VanillaRNN (99.5%) > BLSTM (98.5%) = AttentionRNN (98.5%)

## 2.4 Task 3 — Qualitative Analysis

### 2.4.1 Representative Generated Samples

#### VanillaRNN Samples

Rajesh Neelak Manesh Goram Chandra Amod Woshita  
Nika  
Balnoya Anishni Amarip Farhan Yayuk Akaru Amlan  
Radha  
Albulu Prajin Arusha Akshan Warmal Ajwan Geemant  
Uluka  
Duppis Karasi Ambani Amonya Khavi Gurmanan

**BLSTM Samples**

Aanwar Adleela Grivreeda Agradha Vindar Saita Bushan  
 Tripesh  
 Badni Dirya Raleen Amrisha Tekha Kusan Aya Farmani  
 Ritita Alan Ikjit Yasi Natika Gratini Esha Balkla  
 Bamni Beela Vishi Jayundra Koman Aakrish

**AttentionRNN Samples**

Kamani Laxmina Bandana Sakarya Kanvi Amarthada Orsam  
 Liram  
 Pratini Varvan Shatank Lavna Arshan Virulat Akshin  
 Ojar  
 Amaryan Gurshi Atiti Yastya Jageni Ombata Chaidurath  
 Vijis  
 Wartti Amartsi Indar Urmal Amrit Laxman

**2.4.2 Realism Scorecard**

| Criterion                        | VanillaRNN | BLSTM    | AttentionRNN |
|----------------------------------|------------|----------|--------------|
| Indian phoneme patterns          | Good       | Good     | <b>Best</b>  |
| Prefix structure (A-, Ar-, Dev-) | Good       | Good     | <b>Best</b>  |
| Suffix realism (-esh, -ini, -ya) | Moderate   | Good     | <b>Best</b>  |
| Long-name coherence (8+ chars)   | Weak       | Moderate | <b>Best</b>  |
| Avoids non-Indian clusters       | Moderate   | Moderate | Good         |
| <b>Overall</b>                   |            |          |              |

**Table 2.4:** Qualitative realism assessment across all three models

### 2.4.3 Failure Mode Analysis

| Failure Mode                       | Example               | Model            | Root Cause                              |
|------------------------------------|-----------------------|------------------|-----------------------------------------|
| Unnatural consonant cluster        | Duppis, Geemant       | VanillaRNN       | Sampling from distribution tail         |
| Very short names ( $\leq 3$ chars) | Aya, Nika             | All (rare)       | EOS sampled after first vowel           |
| Non-Indian phoneme pattern         | Grivreedaa, Balkla    | BLSTM            | Backward LSTM distorts shared embedding |
| Repeated character runs            | Gratini, Ritita       | BLSTM            | Attention collapse on single position   |
| Compound stem fusion               | Amarthada, Chaidurath | AttentionRNN     | Attention joins multiple name stems     |
| Abrupt truncation                  | Amod, Ojar            | VanillaRNN, Attn | EOS sampled early in sequence           |

**Table 2.5:** Common failure modes with examples and root causes

### 2.4.4 Why AttentionRNN Works Best

#### Technical Explanation — Attention vs. Vanishing Gradient

The RNN update rule  $h_t = \tanh(W h_{t-1} + U x_t + b)$  compresses all history into a fixed vector at each step. Gradients through this recurrence decay as:

$$\frac{\partial \mathcal{L}_T}{\partial h_1} = \prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}} = \prod_{t=2}^T W^\top \cdot \text{diag}[\tanh'(\cdot)] \rightarrow 0 \quad \text{exponentially}$$

For a 10-character name like “Chandrakesh”, the final character has effectively **no gradient signal** from the first syllable.

The Bahdanau attention context  $\text{ctx} = \sum_s \alpha_s h_s$  creates a **direct retrieval path** to any past state — bypassing sequential compression entirely. When generating “-kesh”, the model can attend directly to  $h_1$  (the ‘C’ step), giving explicit access to the initial consonant cluster. This is why AttentionRNN produces the most phonetically coherent, multi-syllabic names.

### 2.4.5 Architecture Tradeoffs Summary

| Criterion                   | Winner              | Reasoning                                                |
|-----------------------------|---------------------|----------------------------------------------------------|
| Best name quality / realism | <b>AttentionRNN</b> | Attention bypasses vanishing gradient                    |
| Highest novelty             | <b>BLSTM</b>        | BiLM training enriches embeddings; better generalisation |
| Fastest training            | <b>BLSTM</b>        | MPS-optimised LSTM; simplest inference graph             |
| Lowest memory               | <b>VanillaRNN</b>   | 223K params, 0.90 MB — any hardware                      |
| Best diversity              | <b>VanillaRNN</b>   | Most uniform sampling; 99.5% unique                      |
| Best on small data          | <b>AttentionRNN</b> | Attention compensates for limited training signal        |

**Table 2.6:** Final architecture tradeoff analysis

### 3 Quick-Answer Reference Sheet

---

#### Problem 1 — Key Results

##### Corpus size

**0.3356 MB** (335,612 bytes)

##### Top-10 words

lectures(542), course(376), data(363), learning(287), engineering(282), students(268), student(257), design(253), systems(252), computer(248)

##### Best models

CBOW & SkipGram: dim=200, window=2, neg=5

##### Best analogy

SkipGram: undergraduate : degree :: phd → pg  
(sim=0.928)

##### Embedding

research — 200-dimensional vector (CBOW, dim=200)

##### Scratch vs Gensim

CBOW: 0.783 vs 0.998 SkipGram: 0.565 vs 0.974

#### Problem 2 — Key Results

##### Best model

**AttentionRNN** (name quality); **BLSTM** (novelty 94.5%)

##### VanillaRNN

**223,325 parameters, 0.8977 MB**

##### Val losses

VanillaRNN 1.9803  $\approx$  BLSTM 1.9919  $\approx$  AttentionRNN 1.9869

**Novelty** BLSTM (94.5%) > VanillaRNN (93.47%) > AttentionRNN (92.0%)

##### Diversity

VanillaRNN (99.5%) > BLSTM = AttentionRNN (98.5%)